

LABORATÓRIO DE DESENVOLVIMENTO DE APLICAÇÕES MÓVEIS

Engenharia de Software

Tutorial Introdutório de Flutter

Construindo um aplicativo para dividir a conta do restaurante

do formulário ao Figma, e do Figma à IA generativa

Flutter 3.47 (canal stable) · Dart 3.11+ · Material 3

Roteiro de laboratório — nível introdutório · 2 a 4 horas-aula

Setembro de 2026

Sumário

Se o sumário aparecer vazio, clique sobre ele e pressione F9 (Word) ou use Ferramentas → Atualizar → Índices (LibreOffice).

1. Objetivos de aprendizagem	4
Como usar este roteiro	4
2. Preparando o ambiente	6
2.1 Versões usadas neste tutorial	6
2.2 Instalação e verificação	6
2.3 Escolhendo onde rodar	6
3. Especificação e regras de negócio	8
3.1 Entradas	8
3.2 Fórmulas	8
3.3 Exemplo numérico de referência	8
4. Criando o projeto	9
4.1 O que foi gerado	9
4.2 Estrutura que vamos construir	9
5. Passo a passo do código	10
5.1 Passo 1 — A regra de negócio	10
5.2 Passo 2 — Conversão e formatação de números	11
5.3 Passo 3 — O ponto de entrada e o tema	11
5.4 Passo 4 — Um widget de campo reutilizável	12
5.5 Passo 5 — A tela e seu estado	14
5.6 A árvore de widgets resultante	19
6. Executando e depurando	20
6.1 Roteiro de teste manual	20
7. Testes automatizados	21
7.1 Casos de teste sugeridos para a turma	22
8. Problemas frequentes	23
9. Exercícios propostos	23
9.1 Rubrica de avaliação sugerida	23
EXTRA A — Desenhando as telas no Figma	25
A.1 Criando o arquivo e o frame	25
A.2 Definindo os tokens antes de desenhar	25
A.3 Auto Layout: o conceito que traduz para Flutter	26

A.4 Montando a tela, elemento por elemento	27
A.5 As duas telas (estados) e o protótipo	29
A.6 Dev Mode: lendo o design como desenvolvedor	29
A.7 Checklist de entrega do design	29
EXTRA B — Conectando o design a uma IA generativa.....	30
B.1 Panorama dos caminhos.....	30
B.2 Caminho recomendado — servidor MCP do Figma.....	30
B.3 Biblioteca de prompts para a turma.....	31
B.4 Caminho alternativo — Figma Make e Code Layers	32
B.5 Revisão crítica: o que sempre conferir no código gerado	33
B.6 Limites, riscos e uso acadêmico honesto.....	33
B.7 Atividade sugerida para fechar o ciclo.....	33
Referências	35

1. Objetivos de aprendizagem

Ao final deste roteiro de laboratório o aluno deverá ser capaz de:

- Instalar e validar o ambiente Flutter na versão estável mais recente (3.47);
- Criar um projeto Flutter do zero e reconhecer a função de cada pasta e arquivo gerados;
- Diferenciar `StatelessWidget` de `StatefulWidget` e explicar o papel do `setState()` no ciclo de reconstrução da interface;
- Construir um formulário com `Form`, `TextFormField` e validadores, tratando entradas inválidas;
- Separar regra de negócio (Dart puro) da camada de apresentação (widgets), e justificar por que isso facilita o teste automatizado;
- Escrever e executar testes de unidade e de widget;
- Prototipar a mesma tela no Figma usando Auto Layout, variáveis e componentes;
- Conectar o arquivo do Figma a um agente de IA generativa (via servidor MCP) e avaliar criticamente o código produzido.

Aplicativo que será construído

O usuário informa o valor total da conta, quantas pessoas vão dividir e qual a porcentagem de gorjeta do garçom.

O app exibe: (a) o valor da gorjeta, (b) o total a pagar (conta + gorjeta) e (c) quanto cada pessoa paga.

Como usar este roteiro

- **Seções 2 a 4** — preparação do ambiente e criação do projeto. Podem ser feitas antes da aula, como tarefa de casa, para não gastar o laboratório com download de SDK.
- **Seção 5** — o código, escrito em cinco passos. É o núcleo da atividade prática (aproximadamente 90 minutos).
- **Seções 6 a 8** — execução, testes automatizados e os erros mais comuns.
- **Seção 9** — exercícios propostos e rubrica de avaliação sugerida.
- **Extras A e B** — o mesmo aplicativo desenhado no Figma e gerado com auxílio de IA. São opcionais para quem só quer o app funcionando, e recomendados como segunda aula.

Todo o código apresentado é completo e pode ser copiado diretamente. Os comentários dentro do código fazem parte do material didático — peça aos alunos que os leiam em vez de apagá-los.



Figura 1 — Resultado final: à esquerda o estado inicial; à direita, após pressionar “Calcular”.

2. Preparando o ambiente

2.1 Versões usadas neste tutorial

Este roteiro foi escrito e testado com as versões abaixo. Versões diferentes podem exigir pequenos ajustes, mas a lógica permanece a mesma.

Ferramenta	Versão	Observação
Flutter SDK	3.47 (canal stable)	Lançado em 12/08/2026
Dart SDK	3.11 ou superior	Vem embutido no Flutter
Android Studio	2025.2 ou superior	SDK do Android + emulador
Xcode (apenas macOS)	26 ou superior	Necessário só para rodar em iOS
Editor	VS Code + extensões Flutter e Dart	Alternativa: Android Studio

2.2 Instalação e verificação

Baixe o SDK em docs.flutter.dev/install, descompacte em uma pasta sem espaços e sem acentos no caminho (por exemplo, `C:\src\flutter` ou `~/dev/flutter`) e adicione `flutter/bin` ao PATH do sistema. Se já tiver o Flutter instalado, atualize:

```
# Atualiza para a versão estável mais recente
flutter channel stable
flutter upgrade

# Confere as versões instaladas
flutter --version
# Flutter 3.47.0 • channel stable
# Dart 3.11.x • DevTools 2.x.x
```

Em seguida rode o diagnóstico do ambiente. O comando `flutter doctor` verifica cada dependência e sugere a correção de cada problema encontrado:

```
flutter doctor -v
```

Dica para o laboratório

É comum aparecer o item “Android license status unknown”. Resolva com:

```
flutter doctor --android-licenses
```

e aceite todas as licenças. Sem isso o projeto não compila para Android.

2.3 Escolhendo onde rodar

O mesmo código roda em várias plataformas. Para a aula, qualquer uma das opções abaixo serve — a mais rápida para começar é o navegador.

```
# Lista os dispositivos disponíveis (emulador, celular físico, Chrome, desktop)
flutter devices

# Lista e inicia um emulador Android já criado
flutter emulators
```

```
flutter emulators --launch <nome_do_emulador>
```

3. Especificação e regras de negócio

Antes de escrever qualquer widget, é preciso deixar as regras explícitas. Esta é a parte do problema que não depende de Flutter e que, por isso, será implementada em Dart puro.

3.1 Entradas

Entrada	Tipo	Restrição
Valor total da conta	double	Maior que zero
Quantidade de pessoas	int	Inteiro maior ou igual a 1
Percentual da gorjeta	double	Entre 0 e 100

3.2 Fórmulas

```
gorjeta      = valorConta * (percentualGorjeta / 100)
totalAPagar  = valorConta + gorjeta
porPessoa    = totalAPagar / quantidadePessoas
```

Decisão de projeto importante

A gorjeta é dividida entre todas as pessoas — por isso “porPessoa” usa o total já com a gorjeta, e não apenas o valor da conta. Deixe isso claro para a turma: é a origem do erro mais comum nesta atividade (dividir a conta e depois somar a gorjeta inteira a cada pessoa).

3.3 Exemplo numérico de referência

Usaremos este caso como teste manual e, mais adiante, como teste automatizado:

Grandeza	Cálculo	Resultado
Valor da conta	informado pelo usuário	R\$ 250,00
Pessoas	informado pelo usuário	4
Gorjeta	10%	—
Parte do garçom	$250 \times 0,10$	R\$ 25,00
Total a pagar	$250 + 25$	R\$ 275,00
Cada pessoa paga	$275 \div 4$	R\$ 68,75

4. Criando o projeto

Abra o terminal na pasta onde ficam seus projetos e execute:

```
flutter create --org br.edu.eng_software --project-name divisor_conta divisor_conta
cd divisor_conta
flutter run
```

O parâmetro `--org` define o identificador do pacote (`br.edu.eng_software.divisor_conta`), usado nas lojas de aplicativos. O nome do projeto deve estar em **snake_case** — nomes com maiúsculas ou hífen são rejeitados pelo Dart.

4.1 O que foi gerado

Caminho	Para que serve
<code>lib/</code>	Todo o código Dart do aplicativo. É onde vamos trabalhar.
<code>lib/main.dart</code>	Ponto de entrada: contém a função <code>main()</code> .
<code>pubspec.yaml</code>	Manifesto do projeto: nome, versão, dependências e assets.
<code>test/</code>	Testes automatizados.
<code>android/</code> , <code>ios/</code> , <code>web/</code> , <code>linux/</code> , <code>macos/</code> , <code>windows/</code>	Projetos nativos de cada plataforma. Raramente editados no início.
<code>analysis_options.yaml</code>	Regras do analisador estático (lints).
<code>.dart_tool/</code> , <code>build/</code>	Arquivos gerados. Não versionar no Git.

4.2 Estrutura que vamos construir

Em vez de deixar tudo em um único arquivo, vamos organizar o projeto em camadas. Essa separação é o que permite testar a regra de negócio sem abrir a interface.

```
divisor_conta/
├── lib/
│   ├── main.dart                # configuração do app e tema
│   ├── models/
│   │   └── divisao_conta.dart    # regra de negócio (Dart puro)
│   ├── screens/
│   │   └── divisor_home_page.dart # a tela e seu estado
│   ├── utils/
│   │   └── formatador.dart       # conversão texto <-> número
│   └── widgets/
│       └── campo_numerico.dart    # campo de entrada reutilizável
├── test/
│   └── divisao_conta_test.dart    # testes de unidade
└── pubspec.yaml
```

Crie as pastas agora:

```
mkdir -p lib/models lib/screens lib/utils lib/widgets
```

5. Passo a passo do código

Construiremos o aplicativo de dentro para fora: primeiro a regra de negócio, depois os utilitários, depois os widgets e por fim a tela que junta tudo. Copie cada arquivo integralmente e leia os comentários — eles explicam o “porquê” de cada decisão.

5.1 Passo 1 — A regra de negócio

Esta classe não importa nada do Flutter. É Dart puro. Repare que ela não tem métodos `calcular()`: os três valores são expostos como **getters**, calculados sob demanda a partir dos dados de entrada. Assim é impossível existir um objeto com resultados “desatualizados”.

► `lib/models/divisao_conta.dart`

```
/// Modelo de domínio: representa uma conta que será dividida entre
/// várias pessoas, com uma gorjeta (comissão do garçom) opcional.
///
/// Esta classe é Dart puro: NÃO importa nada do Flutter.
/// Isso permite testá-la de forma isolada, sem precisar de interface.
class DivisaoConta {
  /// Valor total da conta, sem a gorjeta. Ex.: 250.00
  final double valorConta;

  /// Quantidade de pessoas que vão dividir a conta. Ex.: 4
  final int quantidadePessoas;

  /// Percentual da gorjeta sobre o valor da conta. Ex.: 10 (para 10%)
  final double percentualGorjeta;

  const DivisaoConta({
    required this.valorConta,
    required this.quantidadePessoas,
    required this.percentualGorjeta,
  }) : assert(valorConta >= 0, 'O valor da conta não pode ser negativo'),
        assert(quantidadePessoas > 0, 'É preciso ao menos 1 pessoa'),
        assert(percentualGorjeta >= 0, 'A gorjeta não pode ser negativa');

  /// Parte que cabe ao garçom.
  double get valorGorjeta => valorConta * (percentualGorjeta / 100);

  /// Conta + gorjeta: o valor que sai do bolso do grupo.
  double get valorTotalAPagar => valorConta + valorGorjeta;

  /// Quanto cada pessoa paga (já incluindo a parte da gorjeta).
  double get valorPorPessoa => valorTotalAPagar / quantidadePessoas;
}
```

Conceitos de Dart aparecendo aqui

`final` — o campo é atribuído uma única vez, no construtor.

`required` — o parâmetro nomeado é obrigatório (null safety).

`assert` — só é avaliado em modo debug; documenta e protege as pré-condições.

`get x => expressão` — getter de uma linha (arrow syntax).

5.2 Passo 2 — Conversão e formatação de números

Todo campo de texto no Flutter devolve uma `String`. Precisamos converter isso em número — aceitando tanto `1234.50` quanto `1.234,50` — e, no caminho inverso, transformar um `double` em texto no formato brasileiro.

► `lib/utils/formatador.dart`

```
/// Converte um double em texto no formato monetário brasileiro.
/// Ex.: 1234.5 -> "R$ 1.234,50"
String formatarReal(double valor) {
  final partes = valor.toStringAsFixed(2).split('.');
  final inteiros = partes[0].replaceAllMapped(
    RegExp(r'\B(?=(\d{3})+(?!\d))'),
    (_) => '.',
  );
  return 'R\$ $inteiros,${partes[1]}';
}

/// Converte o texto digitado pelo usuário em double.
/// Aceita tanto "1234.50" quanto "1.234,50" ou "1234,50".
/// Retorna null quando o texto não representa um número válido.
double? paraDouble(String texto) {
  var limpo = texto.trim();
  if (limpo.isEmpty) return null;
  // Se houver vírgula, ela é o separador decimal e o ponto é de milhar.
  if (limpo.contains(',')) {
    limpo = limpo.replaceAll('.', '').replaceAll(',', '.');
  }
  return double.tryParse(limpo);
}
```

Atenção à sintaxe

Dentro de uma string do Dart, o cifrão inicia interpolação. Para escrever um R\$ literal é preciso escapá-lo: `'R\$ ...'`. Esquecer a barra invertida é erro de compilação. Retornar `double?` (com interrogação) significa "pode ser nulo" — o chamador é obrigado a tratar essa possibilidade. É o null safety do Dart trabalhando a seu favor.

5.3 Passo 3 — O ponto de entrada e o tema

A função `main()` chama `runApp()`, que “pendura” a árvore de widgets na tela. `MaterialApp` fornece tema, navegação e as convenções do Material Design. A partir do Flutter 3.16 o Material 3 é o padrão — não é mais necessário informar `useMaterial3: true`.

► `lib/main.dart`

```
import 'package:flutter/material.dart';

import 'screens/divisor_home_page.dart';

void main() {
  runApp(const DivisorDeContaApp());
}

class DivisorDeContaApp extends StatelessWidget {
  const DivisorDeContaApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```

        title: 'Divisor de Conta',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF00695C)),
        ),
        home: const DivisorHomePage(),
      );
    }
  }
}

```

`ColorScheme.fromSeed` gera uma paleta inteira (primária, superfície, contraste, estados) a partir de uma única cor semente. Trocar o valor `0xFF00695C` muda a identidade visual do app inteiro — um bom experimento para a turma.

5.4 Passo 4 — Um widget de campo reutilizável

Os três campos de entrada só diferem em rótulo, ícone, sufixo e regra de validação. Extrair um widget evita repetir o mesmo bloco três vezes e é a primeira lição prática de composição em Flutter.

► `lib/widgets/campo_numerico.dart`

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

/// Campo de texto reutilizável para entrada de números.
///
/// Encapsula o teclado numérico, o ícone, o rótulo e o validador,
/// evitando repetir o mesmo bloco de código três vezes na tela.
class CampoNumerico extends StatelessWidget {
  const CampoNumerico({
    super.key,
    required this.controller,
    required this.rotulo,
    required this.icone,
    required this.validador,
    this.sufixo,
    this.apenasInteiros = false,
    this.aoAlterar,
  });

  final TextEditingController controller;
  final String rotulo;
  final IconData icone;
  final String? Function(String?) validador;
  final String? sufixo;
  final bool apenasInteiros;
  final void Function(String)? aoAlterar;

  @override
  Widget build(BuildContext context) {
    return TextFormField(
      controller: controller,
      keyboardType: TextInputType.numberWithOptions(decimal: !apenasInteiros),
      inputFormatters: [
        FilteringTextInputFormatter.allow(
          apenasInteiros ? RegExp(r'[0-9]') : RegExp(r'[0-9.,]'),
        ),
      ],
      textInputAction: TextInputAction.next,
      decoration: InputDecoration(
        labelText: rotulo,
        prefixIcon: Icon(icone),
        suffixText: sufixo,
        border: const OutlineInputBorder(),
      ),
    );
  }
}

```

```

    ),
    validator: validator,
    onChanged: aoAlterar,
  );
}
}

```

Propriedade	Efeito
<code>keyboardType</code>	Define qual teclado o sistema mostra. <code>numberWithOptions(decimal: true)</code> inclui vírgula/ponto.
<code>inputFormatters</code>	Filtra o que pode ser digitado. Bloqueia letras antes mesmo da validação.
<code>textInputAction</code>	<code>next</code> faz o botão do teclado pular para o próximo campo.
<code>decoration</code>	Aparência: rótulo flutuante, ícone, sufixo e borda.
<code>validator</code>	Função chamada por <code>Form.validate()</code> . Retorna <code>null</code> se válido, ou a mensagem de erro.

5.5 Passo 5 — A tela e seu estado

Este é o coração do tutorial. `DivisorHomePage` é um `StatefulWidget` porque a tela muda ao longo do tempo: os campos são preenchidos e o cartão de resultado aparece. O widget em si continua imutável — quem guarda os dados mutáveis é a classe `_DivisorHomePageState`.

5.5.1 Esqueleto, controllers e ciclo de vida

► `lib/screens/divisor_home_page.dart`

```
import 'package:flutter/material.dart';

import '../models/divisao_conta.dart';
import '../utils/formatador.dart';
import '../widgets/campo_numerico.dart';

class DivisorHomePage extends StatefulWidget {
  const DivisorHomePage({super.key});

  @override
  State<DivisorHomePage> createState() => _DivisorHomePageState();
}

class _DivisorHomePageState extends State<DivisorHomePage> {
  // Chave que dá acesso ao estado do formulário (para validar tudo de uma vez).
  final _chaveFormulario = GlobalKey<FormState>();

  // Controllers leem e escrevem o texto de cada campo.
  final _controllerConta = TextEditingController();
  final _controllerPessoas = TextEditingController(text: '2');
  final _controllerGorjeta = TextEditingController(text: '10');

  // Guarda o resultado do último cálculo. Null = ainda não calculou.
  DivisaoConta? _resultado;

  @override
  void dispose() {
    // Libera os recursos dos controllers quando a tela sai da árvore.
    _controllerConta.dispose();
    _controllerPessoas.dispose();
    _controllerGorjeta.dispose();
    super.dispose();
  }
}
```

Erro clássico de laboratório

Esquecer o `dispose()` dos `TextEditingController` provoca vazamento de memória. O analisador do Flutter avisa, mas o app continua rodando — por isso o erro passa despercebido. Sempre que criar um controller, crie também o seu `dispose()`.

5.5.2 O método que calcula

Aqui está o ponto central: `setState()` não “atualiza a tela”. Ele marca o State como sujo e agenda uma nova chamada de `build()`. É essa nova execução do `build()` que produz a interface atualizada. Por isso qualquer alteração de dado que precise aparecer na tela deve estar *dentro* do `setState()`.

```
void _calcular() {
  // validate() dispara o validator de cada campo do formulário.
  if (!_chaveFormulario.currentState!.validate()) return;
```

```

// Esconde o teclado depois de calcular.
FocusScope.of(context).unfocus();

final conta = paraDouble(_controllerConta.text!);
final pessoas = int.parse(_controllerPessoas.text.trim());
final gorjeta = paraDouble(_controllerGorjeta.text!);

// setState avisa o Flutter que o estado mudou e a tela deve ser
// reconstruída com os novos valores.
setState(() {
  _resultado = DivisaoConta(
    valorConta: conta,
    quantidadePessoas: pessoas,
    percentualGorjeta: gorjeta,
  );
});
}

void _limpar() {
  _chaveFormulario.currentState!.reset();
  _controllerConta.clear();
  _controllerPessoas.text = '2';
  _controllerGorjeta.text = '10';
  setState(() => _resultado = null);
}

```

5.5.3 Os validadores

Cada validador recebe o texto do campo e devolve **null** quando o valor é aceitável, ou uma mensagem de erro que o Flutter exibe em vermelho abaixo do campo.

```

// -----
// Validadores
// -----

String? _validarConta(String? valor) {
  final numero = paraDouble(valor ?? '');
  if (numero == null) return 'Informe o valor da conta';
  if (numero <= 0) return 'O valor deve ser maior que zero';
  return null; // null significa "campo válido"
}

String? _validarPessoas(String? valor) {
  final numero = int.tryParse((valor ?? '').trim());
  if (numero == null) return 'Informe a quantidade de pessoas';
  if (numero < 1) return 'Deve haver pelo menos 1 pessoa';
  if (numero > 100) return 'Máximo de 100 pessoas';
  return null;
}

String? _validarGorjeta(String? valor) {
  final numero = paraDouble(valor ?? '');
  if (numero == null) return 'Informe a porcentagem (use 0 se não houver)';
  if (numero < 0) return 'A gorjeta não pode ser negativa';
  if (numero > 100) return 'A gorjeta não pode passar de 100%';
  return null;
}

```

5.5.4 O método build

Repare no **SingleChildScrollView**: sem ele, quando o teclado abre, o espaço vertical diminui e o Flutter reclama de **RenderFlex overflow**. Repare também na linha **if (_resultado != null)**

dentro da lista de filhos — é a **renderização condicional**: o cartão simplesmente não existe na árvore enquanto não houver resultado.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Divisor de Conta'),
      actions: [
        IconButton(
          onPressed: _limpar,
          icon: const Icon(Icons.refresh),
          tooltip: 'Limpar',
        ),
      ],
    ),
    body: SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _chaveFormulario,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              CampoNumerico(
                controller: _controllerConta,
                rotulo: 'Valor total da conta',
                icone: Icons.receipt_long,
                sufixo: 'R\$ ',
                validador: _validarConta,
              ),
              const SizedBox(height: 16),
              CampoNumerico(
                controller: _controllerPessoas,
                rotulo: 'Quantidade de pessoas',
                icone: Icons.groups,
                apenasInteiros: true,
                validador: _validarPessoas,
              ),
              const SizedBox(height: 16),
              CampoNumerico(
                controller: _controllerGorjeta,
                rotulo: 'Gorjeta do garçom',
                icone: Icons.room_service,
                sufixo: '%',
                validador: _validarGorjeta,
              ),
              const SizedBox(height: 24),
              FilledButton.icon(
                onPressed: _calcular,
                icon: const Icon(Icons.calculate),
                label: const Text('Calcular'),
                style: FilledButton.styleFrom(
                  padding: const EdgeInsets.symmetric(vertical: 16),
                ),
              ),
              const SizedBox(height: 24),
              // Renderização condicional: só mostra o cartão de
              // resultado depois que o usuário calcula.
              if (_resultado != null) _CartaoResultado(divisao: _resultado!),
            ],
          ),
        ),
      ),
    ),
  );
}
```

5.5.5 O cartão de resultado

Estes dois widgets ficam no **mesmo arquivo**, logo após a classe de estado. O underline no início do nome (_CartaoResultado) os torna privados ao arquivo — uma convenção do Dart para widgets auxiliares que não serão reaproveitados fora dali.

```
/// Cartão que exibe os três valores calculados.
class _CartaoResultado extends StatelessWidget {
  const _CartaoResultado({required this.divisao});

  final DivisaoConta divisao;

  @override
  Widget build(BuildContext context) {
    final tema = Theme.of(context);

    return Card(
      elevation: 0,
      color: tema.colorScheme.surfaceContainerHighest,
      child: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('Resultado', style: tema.textTheme.titleMedium),
            const Divider(height: 24),
            _LinhaResultado(
              rotulo: 'Parte do garçom',
              valor: formatarReal(divisao.valorGorjeta),
            ),
            const SizedBox(height: 12),
            _LinhaResultado(
              rotulo: 'Total a pagar (conta + gorjeta)',
              valor: formatarReal(divisao.valorTotalAPagar),
            ),
            const SizedBox(height: 12),
            _LinhaResultado(
              rotulo: 'Cada pessoa paga',
              valor: formatarReal(divisao.valorPorPessoa),
              destaque: true,
            ),
          ],
        ),
      ),
    );
  }
}

class _LinhaResultado extends StatelessWidget {
  const _LinhaResultado({
    required this.rotulo,
    required this.valor,
    this.destaque = false,
  });

  final String rotulo;
  final String valor;
  final bool destaque;

  @override
  Widget build(BuildContext context) {
    final tema = Theme.of(context);
    final estiloValor = destaque
      ? tema.textTheme.headlineSmall?.copyWith(
```

```

        color: tema.colorScheme.primary,
        fontWeight: FontWeight.bold,
      )
      : tema.textTheme.titleMedium;

return Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  crossAxisAlignment: CrossAxisAlignment.end,
  children: [
    Flexible(child: Text(rotulo, style: tema.textTheme.bodyMedium)),
    const SizedBox(width: 12),
    Text(valor, style: estiloValor),
  ],
);
}
}

```

`Theme.of(context)` busca o tema mais próximo subindo a árvore de widgets. Usar `tema.colorScheme.primary` em vez de uma cor fixa faz o app respeitar automaticamente a paleta e o modo escuro.

5.6 A árvore de widgets resultante

Vale desenhar isso no quadro: entender que a interface é uma árvore de objetos descartáveis, reconstruída a cada mudança de estado, é o conceito que destrava o Flutter.

Árvore de widgets do app Divisor de Conta



Figura 2 — Árvore de widgets do aplicativo.

6. Executando e depurando

```
flutter run          # roda no dispositivo selecionado
flutter run -d chrome # roda no navegador
```

Com o app rodando, o terminal aceita atalhos que valem uma demonstração em aula:

Tecla	Ação	Quando usar
r	Hot reload	Mudou o layout ou um texto. O estado atual é preservado — os campos continuam preenchidos.
R	Hot restart	Mudou <code>main()</code> , o tema, ou o estado inicial. Reinicia do zero.
P	Grade de depuração	Mostra as bordas de cada widget — ótimo para entender espaçamentos.
v	Abre o DevTools	Inspetor de widgets, performance, memória.
q	Encerra	—

6.1 Roteiro de teste manual

1. Abra o app. O cartão de resultado não deve aparecer.
2. Pressione “Calcular” com o campo do valor vazio. Deve surgir “Informe o valor da conta” em vermelho.
3. Digite 250, 4 pessoas e 10%. Pressione “Calcular”. Deve exibir R\$ 25,00 · R\$ 275,00 · R\$ 68,75.
4. Troque a gorjeta para 0 e recalcule: R\$ 0,00 · R\$ 250,00 · R\$ 62,50.
5. Digite 1 pessoa: o valor por pessoa deve ser igual ao total.
6. Digite 3 pessoas com conta 100 e gorjeta 0: R\$ 33,33 (arredondamento para baixo — discuta com a turma o que fazer com o centavo que sobra).
7. Gire o dispositivo e abra o teclado: a tela deve rolar sem erro de overflow.

7. Testes automatizados

Como a regra de negócio está isolada em Dart puro, testá-la é trivial — e rápido, porque não sobe nenhuma interface.

► test/divisao_conta_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:divisor_conta/models/divisao_conta.dart';

void main() {
  group('DivisaoConta', () {
    test('calcula gorjeta, total e valor por pessoa', () {
      const d = DivisaoConta(
        valorConta: 250,
        quantidadePessoas: 4,
        percentualGorjeta: 10,
      );

      expect(d.valorGorjeta, closeTo(25.0, 0.001));
      expect(d.valorTotalAPagar, closeTo(275.0, 0.001));
      expect(d.valorPorPessoa, closeTo(68.75, 0.001));
    });

    test('gorjeta zero não altera o total', () {
      const d = DivisaoConta(
        valorConta: 100,
        quantidadePessoas: 3,
        percentualGorjeta: 0,
      );

      expect(d.valorGorjeta, 0);
      expect(d.valorTotalAPagar, 100);
      expect(d.valorPorPessoa, closeTo(33.333, 0.001));
    });

    test('uma pessoa paga o total', () {
      const d = DivisaoConta(
        valorConta: 80,
        quantidadePessoas: 1,
        percentualGorjeta: 10,
      );

      expect(d.valorPorPessoa, closeTo(d.valorTotalAPagar, 0.001));
    });
  });
}
```

E um teste de widget, que sobe a interface em memória e simula a interação do usuário:

► test/tela_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:divisor_conta/screens/divisor_home_page.dart';

void main() {
  testWidgets('exibe o resultado após preencher e calcular', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(home: DivisorHomePage()),
    );

    // Antes de calcular, o cartão não existe.
    expect(find.text('Resultado'), findsNothing);
  });
}
```

```
// Preenche o primeiro campo (valor da conta).
await tester.enterText(find.byType(TextFormField).first, '250');

await tester.tap(find.text('Calcular'));
await tester.pumpAndSettle();

expect(find.text('Resultado'), findsOneWidget);
expect(find.text('R\$ 25,00'), findsOneWidget);
expect(find.text('R\$ 275,00'), findsOneWidget);
expect(find.text('R\$ 137,50'), findsOneWidget); // 2 pessoas (valor padrão)
});
}
```

Execute todos os testes:

```
flutter test
flutter test --coverage # gera coverage/lcov.info
```

7.1 Casos de teste sugeridos para a turma

Conta	Pessoas	Gorjeta	Garçom	Total	Por pessoa
250,00	4	10%	25,00	275,00	68,75
100,00	3	0%	0,00	100,00	33,33
80,00	1	10%	8,00	88,00	88,00
199,90	5	12%	23,99	223,89	44,78
1.000,00	8	15%	150,00	1.150,00	143,75

8. Problemas frequentes

Sintoma	Causa provável	Correção
RenderFlex overflowed by N pixels	A <code>Column</code> não cabe na tela com o teclado aberto	Envolver em <code>SingleChildScrollView</code> (já feito neste tutorial)
Null check operator used on a null value	<code>currentState!</code> acessado antes do <code>Form</code> existir	Garantir que a <code>GlobalKey</code> está associada ao <code>Form</code> no <code>build</code>
O resultado não muda ao recalcular	Alteração feita fora do <code>setState</code>	Mover a atribuição para dentro do <code>setState(() { ... })</code>
Erro de compilação em ' <code>R\$...</code> '	Cifrão não escapado dentro da string	Escrever ' <code>R\\$...</code> '
A value of type 'String' can't be assigned to 'double'	Uso direto do texto do controller	Converter com <code>paraDouble()</code> / <code>int.parse()</code>
Teclado sem vírgula no iOS	<code>keyboardType</code> sem <code>decimal: true</code>	Usar <code>TextInputType.numberWithOptions(decimal: true)</code>
<code>flutter run</code> não encontra dispositivo	Emulador desligado ou cabo sem depuração USB	<code>flutter devices, flutter emulators -- launch ...</code>

9. Exercícios propostos

Sugestão de continuidade para a próxima aula, em ordem crescente de dificuldade:

1. **Fácil** — substitua o campo de gorjeta por um `Slider` de 0 a 25%, mostrando o percentual escolhido ao lado.
2. **Fácil** — adicione botões rápidos (chips) de 5%, 10% e 15% usando `ChoiceChip`.
3. **Médio** — faça o resultado atualizar em tempo real, sem botão, usando o `onChanged` dos campos.
4. **Médio** — troque o formatador manual pelo pacote `intl` (`flutter pub add intl`) e use `NumberFormat.currency(locale: 'pt_BR')`.
5. **Médio** — trate o centavo residual: se $100 \div 3$ dá 33,33, alguém precisa pagar 33,34. Implemente e teste.
6. **Difícil** — permita divisão desigual (cada pessoa com um peso) usando uma `ListView.builder`.
7. **Difícil** — persista o histórico dos últimos cálculos com `shared_preferences` e exiba em uma segunda tela via `Navigator.push`.

9.1 Rubrica de avaliação sugerida

Critério	Peso
O app compila e roda sem erros ou warnings do analisador	20%

Critério	Peso
Os três valores são calculados corretamente (conferidos pela tabela de casos)	25%
Validação de entrada com mensagens claras (vazio, zero, negativo, > 100%)	20%
Separação entre regra de negócio e interface	15%
Testes automatizados passando	10%
Qualidade da interface: espaçamento, hierarquia, uso do tema	10%

EXTRA A — Desenhando as telas no Figma

Esta seção é opcional para o funcionamento do app, mas é o que aproxima o laboratório da prática profissional: em equipes reais, a tela é desenhada antes de ser codificada, e o desenho é a fonte da verdade sobre medidas, cores e estados.

O que é necessário

Uma conta gratuita em figma.com já permite fazer todo o desenho desta seção.

Recursos de time (bibliotecas compartilhadas) e o servidor MCP para desktop exigem plano pago — a seção B mostra alternativas gratuitas para a parte de IA.

A.1 Criando o arquivo e o frame

1. Acesse figma.com, faça login e clique em “Design file” para criar um novo arquivo.
2. Renomeie o arquivo para “Divisor de Conta — Lab Mobile”.
3. Pressione **F** (ferramenta Frame). No painel direito escolha um preset de celular — **Android Large (360 × 800)** ou **iPhone 14 & 15 Pro (393 × 852)**. Neste tutorial usaremos **390 × 844**.
4. Renomeie o frame (duplo clique no nome na lista de camadas) para “Tela / Inicial”.

Por que o tamanho importa

O Flutter trabalha em pixels lógicos (dp), não em pixels físicos. Um frame de 390 × 844 no Figma corresponde exatamente a 390 × 844 unidades no Flutter. Por isso, um padding de 16 no Figma vira `EdgeInsets.all(16)` no código — sem conversão nenhuma.

A.2 Definindo os tokens antes de desenhar

Antes de posicionar qualquer retângulo, registre as decisões visuais como variáveis. Isso é o equivalente, no design, a declarar constantes em vez de espalhar números mágicos pelo código.

Variáveis de cor

1. No painel direito, aba “Design”, clique no ícone de variáveis (quatro quadrados) → “Create variable” → “Color”.
2. Crie a coleção “tokens” e, dentro dela, um grupo “cor”.
3. Cadastre as cores da tabela abaixo. Elas correspondem à paleta que o Flutter gera a partir da semente #00695C.

Variável	Valor	Onde é usada	Equivalente no Flutter
cor/primaria	#00695C	AppBar, botão, valor em destaque	<code>colorScheme.primary</code>
cor/sobre-primaria	#FFFFFF	Texto sobre o verde	<code>colorScheme.onPrimary</code>
cor/superficie	#FBFDFC	Fundo da tela	<code>colorScheme.surface</code>

Variável	Valor	Onde é usada	Equivalente no Flutter
cor/superfície-alta	#DDE5E3	Fundo do cartão de resultado	<code>colorScheme.surfaceContainerHighest</code>
cor/contorno	#75807F	Borda dos campos	<code>colorScheme.outline</code>
cor/texto	#1B1F1E	Texto principal	<code>colorScheme.onSurface</code>
cor/erro	#BA1A1A	Mensagens de validação	<code>colorScheme.error</code>

Variáveis de número

Crie também um grupo “espaço” com os valores 4, 8, 12, 16, 20 e 24, e um grupo “raio” com 6, 10 e 20. Usar apenas múltiplos de 4 é a convenção do Material Design e evita o rodapé desalinhado que aparece quando cada aluno escolhe um espaçamento diferente.

Estilos de texto

Estilo	Tamanho / peso	Equivalente no Flutter
titulo/appbar	20 / SemiBold	<code>textTheme.titleLarge</code>
titulo/cartao	16 / SemiBold	<code>textTheme.titleMedium</code>
corpo/rotulo	14 / Regular	<code>textTheme.bodyMedium</code>
valor/destaque	24 / Bold	<code>textTheme.headlineSmall</code>
campo/rotulo-flutuante	12 / Regular	Label do <code>InputDecoration</code>

A.3 Auto Layout: o conceito que traduz para Flutter

O Auto Layout do Figma é praticamente o `Column` e o `Row` do Flutter. Aprender a correspondência abaixo é o que torna o design “implementável”.

No Figma	No Flutter
Auto Layout vertical	<code>Column</code>
Auto Layout horizontal	<code>Row</code>
Espaçamento entre itens (gap)	<code>SizedBox</code> entre os filhos ou <code>spacing</code> :
Padding do container	<code>Padding</code> / <code>EdgeInsets</code>
Alinhamento (packed / space between)	<code>mainAxisAlignment</code>
Resize: Fill container	<code>Expanded</code> ou <code>crossAxisAlignment: stretch</code>
Resize: Hug contents	Tamanho natural do widget
Corner radius	<code>BorderRadius.circular(...)</code>
Componente com variantes	Widget com parâmetros (ex.: <code>CampoNumerico</code>)

Para aplicar: selecione os elementos e pressione **Shift + A**. No painel direito ajuste a direção (vertical), o gap (16) e o padding (16).

A.4 Montando a tela, elemento por elemento

A AppBar

1. Desenhe um retângulo de 390×56 no topo, preencha com `cor/primaria`.
2. Adicione um texto “Divisor de Conta” com o estilo `titulo/appbar` em branco, com 16 de margem à esquerda.
3. Adicione o ícone de atualizar (menu “Assets” → biblioteca de ícones, ou o plugin “Material Symbols”) alinhado à direita.
4. Selecione os três elementos, aplique Auto Layout horizontal com alinhamento “space between” e transforme em componente (Ctrl/Cmd + Alt + K).

O campo de entrada

1. Retângulo de 358×56 , raio `raio/6`, sem preenchimento, contorno de 1,5 px em `cor/contorno`.
2. Dentro dele, um Auto Layout horizontal com gap 10 e padding horizontal 12 contendo: ícone (24×24), texto do valor e texto do sufixo.
3. Adicione o rótulo flutuante: um texto pequeno posicionado sobre a borda superior, com um retângulo de fundo na cor `cor/superficie` atrás dele para “cortar” a linha da borda. É exatamente assim que o `OutlineInputBorder` do Material funciona.
4. Selecione tudo e crie o componente “Campo/Numérico”.

Variantes do campo — os estados da interface

Com o componente selecionado, clique em “Add variant” (ícone de losango com +) no painel direito. Crie a propriedade **Estado** com três valores:

Variante	Aparência	Corresponde no código a
Vazio	Borda cinza 1,5 px, rótulo cinza, sem valor	Campo sem texto
Preenchido	Borda <code>cor/primaria</code> 2 px, rótulo verde, valor preto	Campo com foco ou preenchido
Erro	Borda <code>cor/erro</code> 2 px + texto de erro abaixo	<code>validator</code> retornou mensagem

Ponto que costuma faltar nos trabalhos

A maioria dos alunos desenha apenas o estado “tudo preenchido e bonito”. Um design entregue a um desenvolvedor precisa mostrar também: campo vazio, campo com erro, texto muito longo e o estado antes de qualquer cálculo. São exatamente os casos que o código precisa tratar.

O botão e o cartão de resultado

1. Botão: Auto Layout horizontal, padding 16 vertical, largura “Fill container”, raio **raio/20**, preenchimento **cor/primaria**, ícone + texto em branco. Vire componente “Botão/Primário”.
2. Cartão: Auto Layout vertical, gap 12, padding 20, raio **raio/10**, fundo **cor/superficie-alta**.
3. Dentro do cartão: o título “Resultado”, uma linha divisória de 1 px e três linhas (Auto Layout horizontal, “space between”) com rótulo à esquerda e valor à direita.
4. Na última linha, aplique o estilo **valor/destaque** com **cor/primaria** — é a informação que o usuário realmente veio buscar, e por isso deve ser a maior da tela.

Camadas no Figma — página “Divisor de Conta”		
■	Frame · Tela / Resultado	390×844
■	◆ AppBar	componente
■	Auto Layout · Conteúdo	V, gap 16, pad 16
◆	Campo/Numérico	variante: Preenchido
T	Rótulo	
◇	Ícone	
T	Valor	
T	Sufixo	
◆	Campo/Numérico	Pessoas
◆	Campo/Numérico	Gorjeta
◆	Botão/Primário	“Calcular”
■	Cartão de Resultado	V, gap 12, pad 20
T	Resultado	
■	Linha · Parte do garçom	
■	Linha · Total a pagar	
■	Linha · Cada pessoa paga	

Frame 390 × 844 — Auto Layout vertical

Divisor de Conta

Valor total da contaR\$

Quantidade de pessoas

Gorjeta do garçom%

Calcular

Parte do garçomR\$ 25,00

Total a pagarR\$ 275,00

Cada pessoa pagaR\$ 68,75

Padding 16 em toda a coluna (faixa rosa) · **gap 16** entre os filhos

Cor primária #00695C → variável **cor/primaria**

Cantos 6 (campos) / 20 (botão) / 10 (cartão) → **raio/***

Cada campo é um **componente com variantes** (Vazio · Preenchido · Erro)

Figura 3 — À esquerda, a lista de camadas esperada; à direita, o frame com o padding do Auto Layout destacado.

A.5 As duas telas (estados) e o protótipo

1. Duplique o frame (Ctrl/Cmd + D) e renomeie a cópia para “Tela / Resultado”.
2. No primeiro frame, troque os campos para a variante “Vazio” e apague o cartão de resultado.
3. No segundo, use a variante “Preenchido” e preencha os valores do exemplo: R\$ 25,00 · R\$ 275,00 · R\$ 68,75.
4. Vá para a aba **Prototype**, arraste a bolinha do botão “Calcular” da primeira tela até a segunda.
Interação: `On tap` → `Navigate to` → `Instant`.
5. Pressione o botão de play para testar. Esse protótipo é o que você mostra ao “cliente” antes de escrever uma linha de código.

A.6 Dev Mode: lendo o design como desenvolvedor

Ative o **Dev Mode** no canto superior direito (ou `Shift + D`). Ao selecionar qualquer camada, o painel direito passa a mostrar medidas, cores em hexadecimal, tipografia e o código sugerido. Alguns pontos para a aula:

- Selecione dois elementos ao mesmo tempo para ver a distância exata entre eles.
- O painel “Inspect” oferece CSS, iOS e Android. Não existe saída Flutter nativa — o valor real está nas **medidas e nos tokens**, que você traduz manualmente (ou com a IA, na próxima seção).
- Marque o frame como “Ready for dev” para sinalizar à equipe que o design está estável.
- Exporte o frame como PNG 2× (painel “Export”) — essa imagem será útil na seção B.

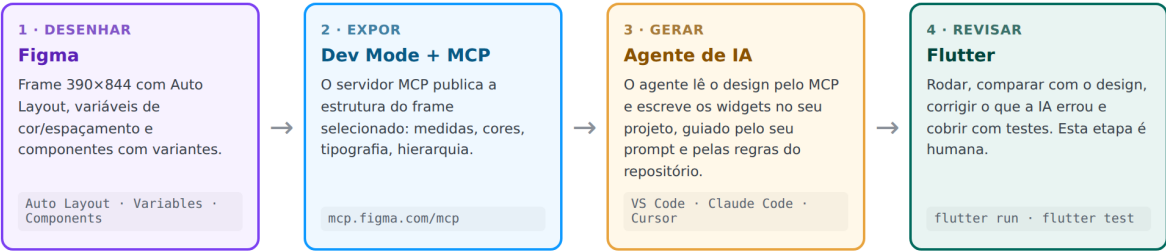
A.7 Checklist de entrega do design

- O frame tem tamanho de dispositivo real e nome descritivo.
- Nenhuma cor foi digitada “na mão”: todas vêm de variáveis.
- Todo agrupamento usa Auto Layout — nada posicionado por coordenada absoluta.
- Campos e botões são componentes, não cópias soltas.
- Os estados vazio, preenchido e erro estão desenhados.
- As camadas têm nomes em português legível (“Cartão de Resultado”), não “Rectangle 27”.
- O protótipo navega da tela inicial para a de resultado.

EXTRA B — Conectando o design a uma IA generativa

A pergunta prática é: como transformar aquele frame do Figma em widgets Flutter sem redigitar cada medida? Há hoje quatro caminhos, com custos e graus de automação diferentes. Comece pelo que sua turma consegue usar sem plano pago.

Do desenho ao código: o fluxo Figma → IA generativa → Flutter



Regra da disciplina: a IA gera o rascunho da camada visual; a regra de negócio, os testes e a revisão são responsabilidade do aluno. Todo código gerado deve ser lido, entendido e citado no relatório da atividade.

Figura 4 — O fluxo completo, do desenho ao código revisado.

B.1 Panorama dos caminhos

Caminho	Como funciona	Requisitos	Indicado para
Servidor MCP do Figma	O agente de IA lê a estrutura do frame diretamente do Figma e escreve o código no seu projeto	Conta Figma + editor com MCP (VS Code, Claude Code, Cursor)	O fluxo mais fiel e o mais próximo do mercado
Figma Make	Descreve-se o app em linguagem natural (ou parte-se de um frame) e o Figma gera um protótipo funcional em código	Plano pago do Figma	Explorar ideias rápido; gera web, não Flutter
Code Layers	Uma camada do canvas vira código interativo dentro do próprio Figma	Recurso lançado no Config 2026, em liberação gradual	Protótipos com comportamento real
Imagem + chat multimodal	Exporta-se o PNG do frame e pede-se o código a um assistente de IA	Nenhum — funciona com conta gratuita	Aula introdutória e trabalho em casa

B.2 Caminho recomendado — servidor MCP do Figma

MCP (*Model Context Protocol*) é um protocolo aberto que permite a um assistente de IA conversar com ferramentas externas. O Figma publica um servidor MCP; ao conectá-lo ao seu editor, o agente deixa de “adivinhar” o design a partir de uma imagem e passa a **ler os dados reais** do frame: medidas em pixels, nomes de variáveis, hierarquia de componentes.

Passo 1 — Habilitar o servidor

Há duas modalidades. A **remota** (<https://mcp.figma.com/mcp>) está disponível em todos os planos e é a mais simples. A **local**, executada pelo aplicativo desktop do Figma, exige assento Dev ou Full em plano pago e é habilitada no menu **Figma → Preferences → Enable local MCP server**.

Passo 2 — Registrar o servidor no editor

No VS Code, crie o arquivo abaixo na raiz do projeto Flutter:

► **.vscode/mcp.json**

```
{
  "servers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

Em ferramentas de linha de comando, o registro costuma ser um único comando:

```
# Exemplo com o Claude Code
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Verificação e observação importante

Na primeira conexão o editor abre o navegador para você autorizar o acesso à sua conta Figma. Depois disso, peça ao agente: “liste as ferramentas do servidor figma”. Se ele responder com nomes como `get_design_context` / `get_screenshot`, a conexão está funcionando. URLs, portas e nomes de ferramentas mudam com frequência — confirme sempre na documentação oficial antes da aula.

Passo 3 — Selecionar o frame e pedir o código

1. No Figma, selecione o frame “Tela / Resultado”.
2. Copie o link da seleção: botão direito → **Copy link to selection**.
3. Cole o link no chat do agente dentro do editor, junto com um prompt bem especificado (próxima subseção).
4. O agente lê o design pelo MCP, cria/edita os arquivos e você revisa o diff antes de aceitar.

B.3 Biblioteca de prompts para a turma

A qualidade do resultado depende quase inteiramente da qualidade do pedido. Compartilhe estes modelos com os alunos e peça que os adaptem.

Prompt 1 — Gerar a tela a partir do frame

Você é um desenvolvedor Flutter sênior. Use o servidor MCP do Figma para ler o frame em <cole aqui o link da seleção>.

Gere um widget Flutter para a versão 3.47 (Material 3) seguindo estas regras:

- Não use pacotes externos; apenas `package:flutter/material.dart`.
- Não use valores de cor fixos: derive tudo de `Theme.of(context).colorScheme`.
- Respeite exatamente os paddings, gaps e raios do Auto Layout do frame.

- Crie um StatelessWidget para o cartão de resultado, recebendo os valores por parâmetro; não coloque lógica de cálculo dentro de widgets.
- O arquivo deve passar em "flutter analyze" sem avisos.

Entregue apenas o conteúdo do arquivo lib/screens/divisor_home_page.dart e liste, ao final, quais medidas você não conseguiu extrair do design.

Prompt 2 — Converter as variáveis do Figma em tema

Leia as variáveis da coleção "tokens" do arquivo do Figma pelo MCP e gere um ThemeData do Flutter com um ColorScheme completo e um TextTheme, mapeando cada variável para o papel semântico correspondente (primary, onPrimary, surface, outline, error...).

Explique em comentários a decisão de mapeamento de cada variável.

Prompt 3 — Revisar a fidelidade do que já existe

Compare o widget em lib/screens/divisor_home_page.dart com o frame do Figma em <link>. Liste em uma tabela apenas as divergências reais de espaçamento, tamanho de fonte, cor e raio, no formato:
elemento | valor no Figma | valor no código | correção sugerida.
Não altere nenhum arquivo ainda.

Prompt 4 — Sem MCP: a partir da imagem exportada

Anexo o PNG de uma tela desenhada no Figma para um app de dividir contas.

Reproduza esta tela em Flutter 3.47 com Material 3. Detalhes que a imagem não mostra e que você deve assumir:

- frame de 390x844; padding de 16 na coluna principal; gap de 16 entre campos
- cor semente do tema: #00695C
- os três campos são TextFormField com OutlineInputBorder e teclado numérico

Não implemente o cálculo – apenas o layout. Marque com // TODO os pontos em que precisou adivinhar alguma medida.

O que faz um prompt funcionar

Papel (“desenvolvedor Flutter sênior”) + versão exata do framework + restrições explícitas (“sem pacotes externos”, “sem cores fixas”) + formato de saída + pedido de que a IA declare suas incertezas. Esse último item é o mais subestimado: obrigar o modelo a listar o que ele adivinhou transforma uma resposta confiante em uma resposta auditável.

B.4 Caminho alternativo — Figma Make e Code Layers

O **Figma Make** gera um aplicativo funcional a partir de uma descrição em linguagem natural ou de um frame existente, e permite continuar editando por prompt. O **Code Layers**, apresentado na Config 2026, vai além: transforma uma camada do canvas em código interativo que roda dentro do próprio Figma, com ida e volta entre design e código.

Limitação a deixar clara para a turma: essas ferramentas produzem, hoje, **código web (HTML/CSS/React)** — não Dart. Em uma disciplina de Flutter elas servem para validar a ideia e o comportamento rapidamente, e o resultado precisa ser reimplementado (ou traduzido com auxílio da IA) em widgets.

B.5 Revisão crítica: o que sempre conferir no código gerado

Esta é a parte pedagogicamente mais importante do extra. Peça que o aluno preencha este checklist no relatório da atividade.

O que verificar	Por que costuma dar errado
O código compila e passa em <code>flutter analyze</code>	Modelos misturam APIs de versões diferentes do Flutter
A aritmética está correta	A IA frequentemente divide a conta e soma a gorjeta inteira a cada pessoa
Os <code>TextEditingController</code> têm <code>dispose()</code>	Omissão silenciosa: não quebra o app, mas vaza memória
Não há cores nem tamanhos “mágicos” fixos no código	O gerado costuma ignorar o tema e cravar <code>Color(0xFF...)</code>
A regra de negócio não está dentro do <code>build()</code>	O código gerado tende a concentrar tudo em um só widget
O layout não estoura com o teclado aberto	Falta o <code>SingleChildScrollView</code>
Os textos estão em português e sem erro	Modelos costumam gerar rótulos em inglês
Há tratamento para entrada inválida	A validação é a primeira coisa que a IA omite

B.6 Limites, riscos e uso acadêmico honesto

- **Fidelidade não é garantia de qualidade.** Um layout pixel-perfect pode ter acessibilidade ruim, contraste insuficiente ou área de toque menor que 48 dp.
- **O contexto é limitado.** Frames muito grandes ou com centenas de camadas costumam ser truncados; peça um componente por vez.
- **Design mal estruturado gera código mal estruturado.** Se o frame não usa Auto Layout nem componentes, a IA devolve widgets aninhados com posições absolutas. A qualidade do design é o principal fator de qualidade do resultado.
- **Confidencialidade.** Conectar o MCP dá ao agente acesso ao conteúdo dos seus arquivos do Figma. Em projetos com dados sensíveis, verifique a política da instituição.
- **Autoria.** Sugestão de regra para a disciplina: o uso de IA é permitido e incentivado, desde que o aluno (a) registre no relatório quais prompts usou, (b) explique o que alterou no código gerado e (c) seja capaz de defender cada linha em arguição oral. Código que o aluno não sabe explicar não é aceito.

B.7 Atividade sugerida para fechar o ciclo

1. Cada dupla desenha a tela no Figma seguindo o Extra A (peso: estrutura, não estética).
2. Gera a primeira versão do código com IA, por um dos caminhos da seção B.
3. Registra o prompt usado e o código recebido, sem edição, em um anexo.
4. Preenche o checklist de revisão (B.5) apontando cada problema encontrado.
5. Corrige o código, implementa a regra de negócio manualmente e faz os testes passarem.

6. Entregue um relatório de uma página comparando “o que a IA acertou” e “o que só um desenvolvedor resolveria”.

Referências

- Flutter — Release notes e novidades da versão 3.47: docs.flutter.dev/release/whats-new
- Flutter — Instalação e primeiros passos: docs.flutter.dev/install
- Flutter — Documentação de Form e validação: docs.flutter.dev/cookbook/forms/validation
- Flutter — Testes de widget: docs.flutter.dev/cookbook/testing/widget/introduction
- Material 3 — Diretrizes de design: m3.material.io
- Figma — Guia do servidor MCP (Dev Mode MCP Server): help.figma.com
- Figma — Blog: “Introducing our Dev Mode MCP server”: figma.com/blog/introducing-figma-mcp-server
- Figma — Config 2026 (Code Layers, Figma Motion, agente): figma.com/blog/config-2026-recap
- Figma — Design to code com Figma Make: figma.com/solutions/design-to-code
- Model Context Protocol — Especificação: modelcontextprotocol.io

Todo o código-fonte apresentado neste tutorial foi executado e validado com a versão do Dart correspondente ao Flutter 3.47 antes da publicação deste documento.